CASE STUDY

TLB Address Translation Simulator: An Interactive Approach to Virtual-to-Physical Translation and TLB Performance Analysis

Submitted by:

P V Sai Likith (24BCI0174)

M V S Sujan (24BCE2294)

Shrijit (24BCE2469)

Rajnihil (24BCT0191)

CONTENTS

1. Project Title 3
2. Introduction 3
3. Problem Statement 4
4. Objectives 5
5. Survey on Existing Work 5
6. Existing and Proposed Systems 6
7. Methodology with Flowchart 8
8. Software and Hardware Requirements 11
9. System Architecture 12
10. Implementation Timeline 14
11. Expected Outcomes 15
12. Future Scope 17
13. Challenges and Ethical Considerations 18
14. Conclusion 19

References 19

1. Project Title

TLB Address Translation Simulator: An Interactive Approach to Virtual-to-Physical Translation and TLB Performance Analysis

The project builds **TLB/Scope**, a browser-based "MMU workbench" that simulates how a processor's memory-management unit (MMU) turns a virtual address into a physical address with the help of a Translation Lookaside Buffer (TLB). The key facts about the project are summarised in Table 1.

Table 1: *Project at a glance*

Field	Details
Project name	TLB/Scope: an interactive MMU workbench
Course	Operating Systems
Topic	Module 5, Memory Management. Mini Project #45: "Simulate virtual-to-physical address translation using TLB"
Project type	Interactive, browser-based educational simulator
Technology	HTML5, CSS3 and JavaScript (no frameworks, no back end); Node.js for automated tests
Deliverables	Ten-page web application, automated test suite (96 engine tests and 29 page tests), printable report

2. Introduction

Modern operating systems use **virtual memory**. Each process sees its own large, contiguous address space, while the physical memory underneath is limited and shared by many processes. With **paging**, both spaces are divided into fixed-size blocks: *pages* in virtual memory and *frames* in physical memory. Every address the CPU issues must be translated from a **virtual address (VA)** to a **physical address (PA)** before memory can be accessed. The VA is split into a **virtual page number (VPN)** and an **offset**. The **page table** maps the VPN to a **physical frame number (PFN)**, and the offset is copied unchanged because a page and a frame are the same size.

The page table itself is stored in main memory. Without extra hardware, every memory reference would need at least one additional memory access to read the page-table entry (PTE), and k additional accesses for a k-level page table. That would at least double the cost of every memory access. To avoid this, the MMU contains a **Translation Lookaside Buffer (TLB)**, a small, fast, associative cache of recently used translations. On a **TLB hit**, the PFN is available in a few nanoseconds. On a **TLB miss**, the page table is walked and the translation is inserted into the TLB. If the PTE is not present, a **page fault** transfers control to the operating system. Programs show strong locality of reference, so a small TLB can serve most accesses. This

makes the TLB one of the most important performance structures in a computer system (Figure 1).

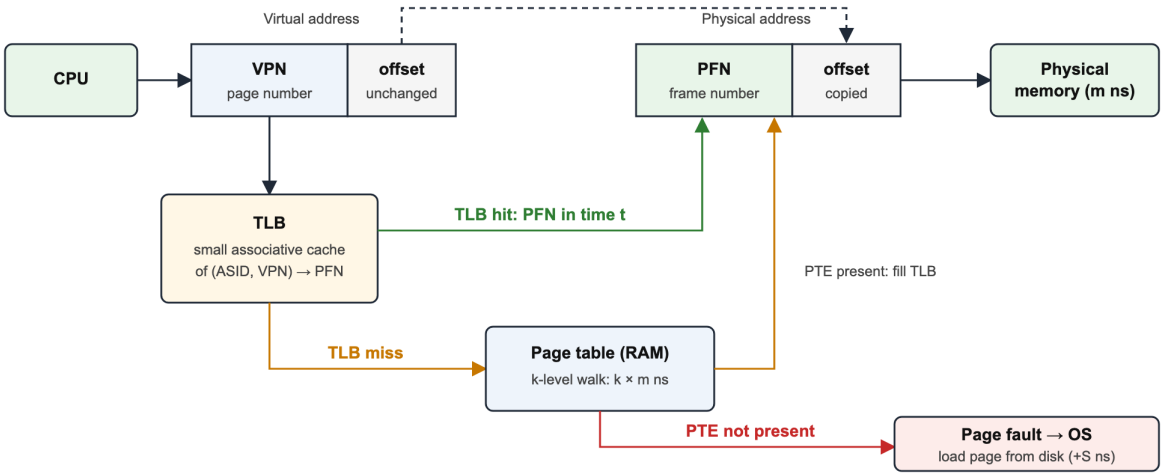


Figure 1: *Virtual-to-physical address translation with a TLB*

These mechanisms are central to the Operating Systems syllabus. However, they happen inside hardware in nanoseconds and cannot be observed directly. This case study presents **TLB/Scope**, an interactive simulator that makes every step of translation visible: the bit-level split of the address, TLB lookup and replacement decisions, page-table walks, page faults, protection checks and the resulting effective access time. Students can also compare different design choices on the same workload.

3. Problem Statement

To design and implement an interactive simulator that models virtual-to-physical address translation through a TLB and a page table, shows every intermediate step of each translation, and measures how the size, organisation and replacement policy of the TLB affect the hit ratio and the effective access time.

The need for such a simulator comes from the following difficulties faced by students and teachers:

- **Translation is invisible.** It happens in hardware within a few nanoseconds, so students only see static textbook diagrams and never watch a real translation step by step.
- **Hand calculation is slow and error-prone.** Splitting addresses into VPN, offset, set index and tag in binary, tracing a replacement policy over a long reference string and computing the effective access time by hand all take time, and mistakes go unnoticed.
- **Design choices are hard to compare.** The effect of TLB size, associativity, replacement policy, page-table depth or address-space identifiers (ASIDs) can only be seen by running the same workload under different configurations.

- **Counter-intuitive behaviour is hard to believe without seeing it.** Examples include Belady's anomaly, thrashing when the working set is larger than the TLB, and conflict misses in a direct-mapped TLB.
- **Existing tools are either too simple or too complex.** Page-replacement animators do not model a TLB, while research simulators such as gem5 need long setup and produce only statistics.

4. Objectives

The objectives of the project are:

1. To show the decomposition of a virtual address into VPN, TLB tag, set index, page-table level indices and offset at bit level, for address widths from 4 to 32 bits.
2. To simulate the complete translation path: TLB lookup, hit or miss, multi-level page-table walk (1 to 4 levels), TLB fill and replacement.
3. To implement the TLB replacement policies FIFO, LRU, LFU, CLOCK (second chance) and seeded RANDOM, with OPT (Belady's optimal algorithm) as a benchmark.
4. To support fully associative, set-associative and direct-mapped TLB organisations.
5. To handle page faults in two modes, *report only* and *demand paging* (with FIFO, LRU or CLOCK frame replacement, dirty write-back and TLB shootdown), and to detect write-protection faults.
6. To model multiple processes with ASID-tagged TLB entries, compared with flushing the TLB on every context switch.
7. To measure the hit ratio, misses, faults, evictions and effective access time (EAT), and to compare the analytical EAT model with the value measured from the run.
8. To provide time travel (rewind, branch and what-if replay), export and sharing, and to verify correctness with an automated test suite.

5. Survey on Existing Work

We studied textbooks, educational simulators, research simulators and visual tools that deal with address translation and page replacement. Table 2 summarises what each offers and why it does not fully solve the problem stated in Section 3.

Table 2: *Survey of existing work*

#	Work / tool	What it offers	Limitation for this problem
1	Silberschatz, Galvin and Gagne, *Operating System Concepts*	Standard textbook treatment of paging, TLBs, EAT and page replacement, with worked numerical examples.	Static figures; students cannot try their own addresses or configurations.

#	Work / tool	What it offers	Limitation for this problem
2	Arpaci-Dusseau, *Operating Systems: Three Easy Pieces* (OSTEP)	Clear chapters on TLBs, ASIDs and multi-level page tables, with Python command-line simulators that generate translation exercises.	Text-based. The paging simulators do not model a TLB; TLB behaviour is studied by measuring real hardware instead.
3	MOSS Memory Management Simulator (companion to Tanenbaum's *Modern Operating Systems*)	Java GUI that runs a list of memory operations over virtual pages and shows page faults and page replacement.	Focuses on paging and page replacement; no TLB, no set associativity, no ASIDs and no EAT measurement.
4	CPU-OS Simulator (B. Mustafa)	Integrated educational simulator of a CPU and a simple operating system, including memory views.	Very broad scope; TLB internals and the comparison of replacement policies are not its focus.
5	gem5 (Binkert et al., 2011)	Full-system research simulator with detailed MMU, TLB and page-table-walker models.	Steep learning curve, long setup and simulation times; output is statistics, not an explanation of each translation.
6	SimpleScalar (Burger and Austin, 1997)	Cycle-level processor simulator with configurable caches and TLBs.	Command-line tool aimed at architecture research; no visualisation of individual translations.
7	Dinero IV (Edler and Hill)	Fast trace-driven simulator of cache hierarchies.	Models caches only; no page tables, page faults or protection.
8	Online page-replacement visualisers	Web pages that animate FIFO, LRU and OPT on a reference string.	Model page frames only; no address split, no TLB and no timing.
9	Hardware performance counters (e.g. Linux *perf* dTLB-load-misses)	Measure real TLB misses of real programs.	Black box: they report counts but not why each miss happened, and need real hardware and Linux.

5.1 Research gap

None of the surveyed works combines all of the following in one tool that runs without installation: (a) a bit-level explanation of each individual translation, (b) a choice of TLB organisation and replacement policy, (c) page faults, protection and multiple processes, and (d) quantitative comparison and automated verification. TLB/Scope is designed to fill this gap.

6. Existing and Proposed Systems

6.1 Existing system

In the existing approach, address translation is taught with textbook figures and board numericals, supported at most by a page-replacement animator or a command-line script. Its main drawbacks are:

- Learning is passive: the student cannot change the address, the page size or the TLB and see the effect.
- Every numerical is solved by hand, so long traces and multi-level page tables are rarely attempted.
- The TLB, the page table and page replacement are taught separately, so the link between a TLB miss, a page-table walk and a page fault is not obvious.
- There is no way to measure the EAT of a real trace or to compare policies on the same input.
- Errors in manual calculations go unnoticed because there is nothing to check the answer against.

6.2 Proposed system

The proposed system, **TLB/Scope**, is a browser-based MMU workbench organised as ten connected pages (Home, Machine, Translate, Timeline, Structures, Analytics, Policy lab, Learn, Self-test and Report). Its main features are:

- **Address probe** that accepts decimal, hexadecimal (0x), binary (0b) and octal (0o) input and predicts the outcome (hit, miss or fault) before translating.
- **Bit surgery** view that draws the VA bit by bit: the offset bits drop straight into the PA and the PFN bits are swapped in from the TLB, the page table or the OS.
- **MMU datapath** with six stages (CPU, TLB, WALK, OS, CHK, RAM) and a step-through mode that explains each stage with the actual numbers.
- **Calculation player** that animates every formula, for example floor(300 / 256) = 1 and 1 × 256 + 44 = 300.
- **Logic-analyzer timeline** with rewind, branching and what-if replay of the whole history under a new configuration.
- **Live structures**: the TLB as sets × ways, a clickable page-table map with a PTE inspector, and physical frames coloured by process.
- **Analytics**: eight key indicators, a hit-ratio chart and an EAT calculator comparing the model, the measured run and a system with no TLB.
- **Policy lab**: six policies × seven TLB sizes on the same trace, with automatic detection of Belady's anomaly.
- Ten one-click scenarios, an in-browser self-test page and a printable A4 report.

6.3 Comparison

Table 3: *Existing system versus proposed system*

Aspect	Existing system	Proposed system (TLB/Scope)
Interaction	Static diagrams and hand calculation	Any address and any configuration, with an instant result

Aspect	Existing system	Proposed system (TLB/Scope)
Visibility of steps	Final answer only	Bits, datapath stages, formulas and data structures
TLB organisation	Usually fully associative only	Fully associative, set-associative and direct-mapped
Replacement policies	One at a time, traced by hand	FIFO, LRU, LFU, CLOCK, RANDOM and OPT side by side
Page faults and protection	Discussed separately	Integrated: report-only or demand paging, write protection
Multiple processes	Rarely covered	ASID tagging versus flush on context switch
Performance	EAT formula only	Modelled, measured and no-TLB EAT compared
Correctness	Not checked	96 automated engine tests and invariant checks
Setup	None, or complex installation	Opens in any modern browser; no installation

7. Methodology with Flowchart

7.1 Development approach

The project followed an iterative approach in six phases:

1. **Requirement analysis:** list the syllabus topics (paging, TLB, EAT, page replacement) and the features the simulator must offer.
2. **Model design:** define the machine configuration, the translation formulas and a deterministic engine interface.
3. **Engine implementation:** write a DOM-free JavaScript engine (engine.js) that runs in both the browser and Node.js.
4. **Interface design:** build pages that each do one job and share a single session.
5. **Verification:** hand-derived test cases, invariant checking, mutation testing and browser testing.
6. **Documentation:** learning notes, viva questions, a printable report and this case study.

7.2 Translation model

Let S be the page size, t the TLB lookup time, m the memory access time, k the number of page-table levels, h the TLB hit ratio, p the page-fault rate and F the page-fault service time. The simulator uses the formulas in Table 4.

Table 4: *Formulas used by the simulator*

Quantity	Formula
Offset bits	log2(S)
Virtual page number	VPN = floor(VA / S)
Offset	offset = VA mod S
Number of TLB sets	sets = entries / ways
Set index and tag	set = VPN mod sets; tag = floor(VPN / sets)
Physical address	PA = PFN × S + offset
TLB hit time	t + m
TLB miss time (k-level table)	t + k·m + m
Page-fault time	miss time + F
Effective access time	EAT = h(t + m) + (1 − h)(t + (k + 1)m) + p·F

7.3 Algorithm for one memory access

1. Validate the virtual address and the process number. Invalid input is rejected without changing any state.
2. If the process differs from the current one, record a context switch. In flush mode, empty the TLB.
3. Split the VA into VPN, offset, TLB set index, tag and page-table level indices.
4. Search the selected TLB set for an entry with a matching (ASID, VPN). On a hit, update the policy data (last use, use count, reference bit) and take the PFN from the entry.
5. On a miss, walk the page table (k memory reads). If the PTE is not present, raise a page fault. In report-only mode the fault is recorded and the access stops. In demand-paging mode, a free frame or a victim frame (FIFO, LRU or CLOCK) is chosen; the victim is written back if dirty, its PTE is invalidated, its TLB entry is shot down, and the new page is loaded.
6. Insert the translation into the TLB, evicting a victim chosen by the TLB replacement policy if the set is full.
7. If the access is a write and the page is read-only, raise a protection fault; no memory access takes place.
8. Otherwise form PA = PFN × S + offset, access memory, set the referenced and dirty bits, and update the statistics and the timeline.

Figure 2 shows the same algorithm as a flowchart.

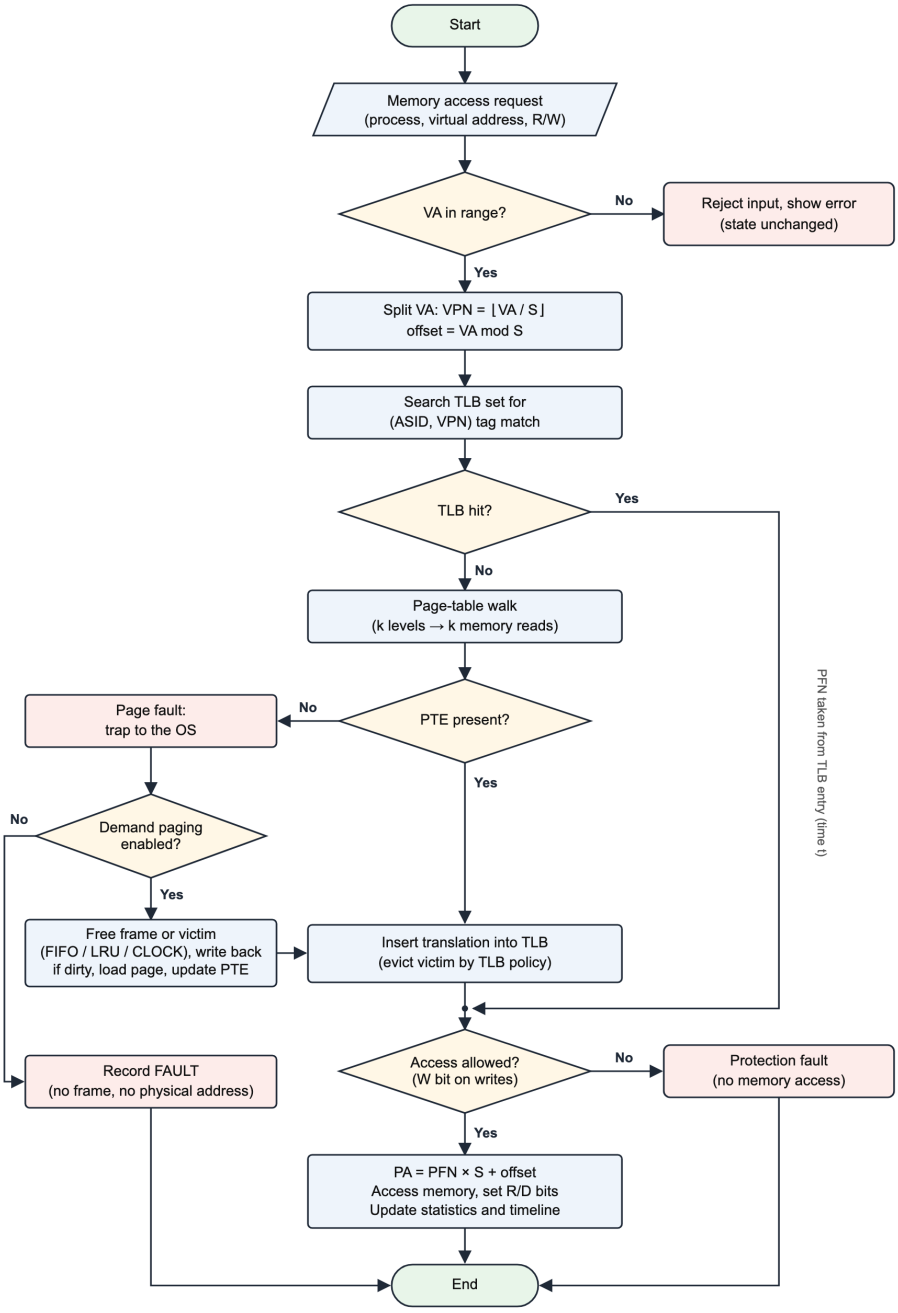


Figure 2: *Flowchart of one memory access in the simulator*

7.4 TLB replacement policies

Table 5: *Replacement policies implemented*

Policy	Victim chosen	Remark
FIFO	Entry inserted earliest	Simple; can show Belady's anomaly
LRU	Entry used least recently	Stack algorithm; never shows Belady's anomaly
LFU	Entry with the fewest uses (ties broken by LRU)	Good for skewed workloads; slow to adapt

Policy	Victim chosen	Remark
CLOCK	First entry with reference bit 0; bits are cleared as the hand passes	Cheap approximation of LRU (second chance)
RANDOM	Pseudo-random entry from a seeded generator	Reproducible because the seed is fixed
OPT	Entry whose next use is farthest in the future	Optimal lower bound; needs the future trace (policy lab only)

7.5 Worked example

With the default machine (16-bit VA, 12-bit PA, 256-byte pages, a 4-entry fully associative LRU TLB, t = 10 ns, m = 100 ns, one-level page table, report-only faults), the following inputs were entered in the running simulator. Table 6 shows the outputs it produced.

Table 6: *Sample input and output from the simulator*

Access	Input VA	VPN, offset	Outcome	PA	Time
1	300	1, 44	TLB miss; page walk finds VPN 1 → PFN 1; translation inserted into TLB	300	10 + 100 + 100 = 210 ns
2	300	1, 44	TLB hit; PFN 1 read from the TLB	300	10 + 100 = 110 ns
3	0x1A2B (6699)	26, 43	Page fault: page 26 is not resident (report-only mode)	none	10 + 100 = 110 ns

For VA 300: VPN = floor(300 / 256) = 1, offset = 300 mod 256 = 44, and PA = 1 × 256 + 44 = 300. With a hit ratio of h = 0.8, the EAT is 0.8 × 110 + 0.2 × 210 = **130 ns**, compared with 200 ns if there were no TLB at all.

8. Software and Hardware Requirements

8.1 Software requirements

Table 7: *Software requirements*

Component	Requirement
Operating system	Windows 10/11, macOS or Linux
Web browser	Recent Google Chrome or Microsoft Edge (recommended, for native page transitions); Firefox and Safari are supported with a fallback animation
Languages	HTML5, CSS3, JavaScript (ES2020)
Test runtime	Node.js 18 or later (optional; runs the automated test suite with npm test)
Local web server	Python 3 (optional; python3 -m http.server 5173). The app also opens directly from index.html
Development tools	Visual Studio Code, Git, Chrome DevTools

Component	Requirement
External libraries	None: no frameworks, no back end and no database

8.2 Hardware requirements

Table 8: *Hardware requirements*

Component	Minimum	Recommended
Processor	Dual-core, 1.6 GHz	Quad-core, 2.0 GHz or better
RAM	4 GB	8 GB
Storage	50 MB free space	100 MB free space
Display	1366 × 768	1920 × 1080 (the layout also adapts to 375-pixel-wide phones)
Network	Not needed after download	Not needed
Input devices	Keyboard and mouse, or touch screen	Keyboard and mouse

9. System Architecture

TLB/Scope uses a layered architecture, shown in Figure 3 and described below. The user works with ten HTML pages. A shared interface shell provides navigation and effects. A session layer stores the history of operations and replays it through a pure simulation engine, which holds all the operating-system and hardware logic.

- **Presentation layer:** ten pages, each doing one job: configure the machine, translate one address, run a workload, inspect structures, analyse performance, compare policies, learn, self-test and print a report.
- **Interface shell:** shell.js draws the tabs, context bar, journey stepper and command palette on every page; fx.js adds animations and the live mini-MMU on the home page; calc.js plays the step-by-step calculation; report.js builds the printable report.
- **Session and controller layer (app.js):** keeps the configuration, the list of operations and the current time cursor, replays them through the engine, and renders the panels of the current page.
- **Simulation engine (engine.js):** a deterministic module with no user-interface code. It performs address decomposition, TLB lookup and replacement, page-table walks, fault handling, protection checks, statistics and EAT, policy comparison and invariant checking.
- **Persistence and sharing:** the session is saved in the browser's localStorage, can be shared as a link whose data stays in the URL fragment, and can be exported as CSV or JSON.

- **Verification:** the same engine code is tested by 96 engine tests (in Node.js and in the browser) and 29 page-level smoke tests.

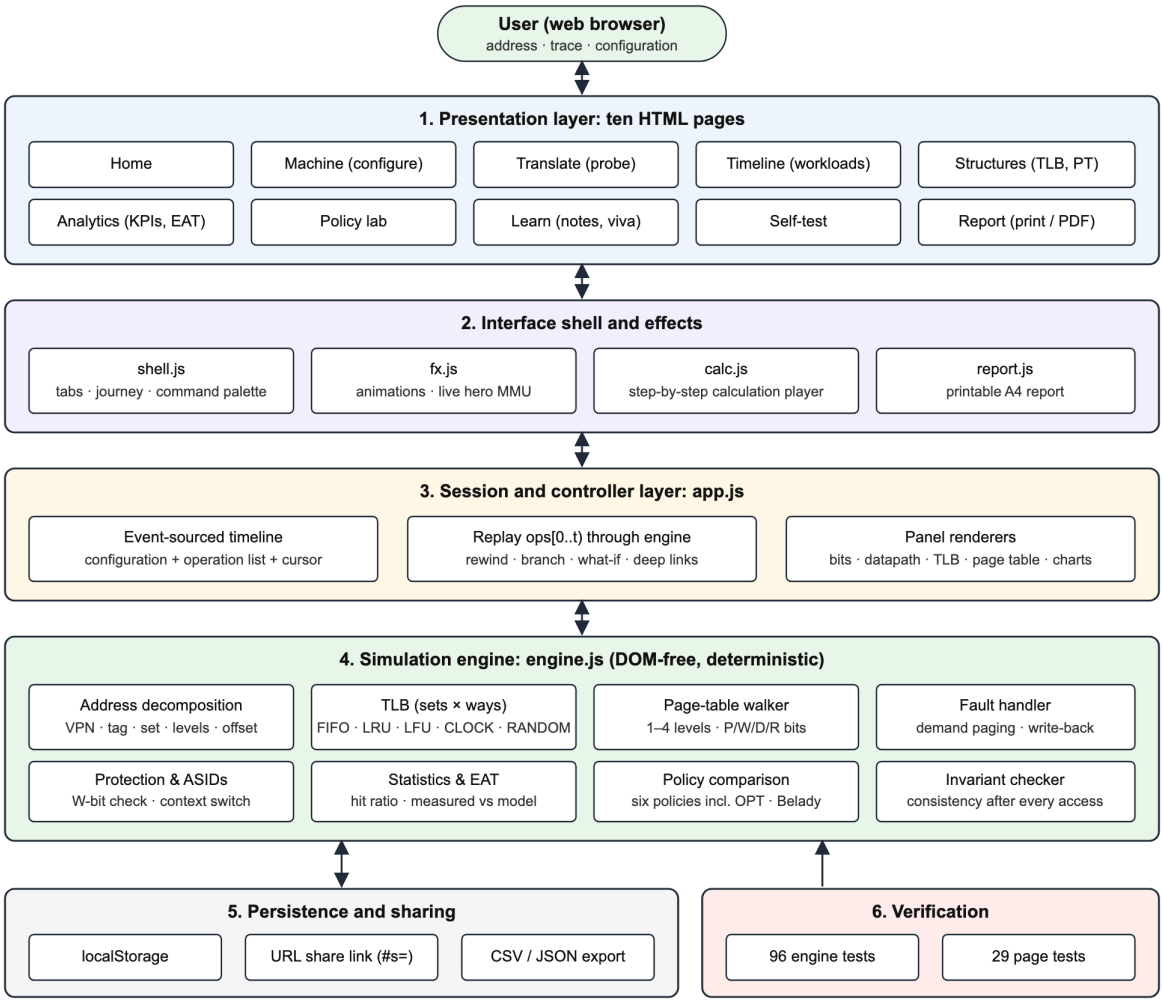


Figure 3: *System architecture of TLB/Scope*

9.1 Key design decision: an event-sourced timeline

The interface never changes the translation state directly. Every user action (an access, a permission change, an INVLPG or a flush) is appended to a list of **operations**. Because the engine is deterministic (even the RANDOM policy uses a seeded generator), the state at any time t is simply the replay of operations 0 to t. This one mechanism provides time travel, branching, what-if replay under a new configuration, and shareable links.

9.2 Data structures

Page tables are stored as typed arrays: an Int32Array of PFNs plus a Uint8Array of flag bytes holding the present, writable, dirty and referenced bits. This keeps even a 32-bit address space with 4 KiB pages (2^20 entries) fast. The TLB is an array of sets, each holding an array of ways. Every physical frame records its owner (process, VPN), which acts as an inverted page table so that a victim's TLB entry can be invalidated in constant time.

Table 9: *Source modules*

File	Responsibility	Lines
js/engine.js	DOM-free MMU engine shared by the browser and Node.js	721
js/app.js	Shared session, replay, persistence, deep links and panel renderers	1,406
js/calc.js	Animated calculation player (Translate page)	398
js/fx.js	Effects, live hero MMU and command palette	318
js/shell.js	Tabs, context bar, journey and page transitions	246
js/report.js	Printable report (Report page)	105
tests/suite.js, run.js, pages.js	96 engine tests and 29 page-level smoke tests	-

10. Implementation Timeline

The project was planned over twelve weeks, as shown in Table 10.

Table 10: *Implementation timeline*

Week	Phase	Activities	Deliverable
1	Requirement analysis	Study the syllabus, survey existing tools, define the feature list	Problem statement and requirements
2	Model design	Define configuration, formulas, engine interface and test plan	Design document
3–4	Core engine	Address split, TLB, page-table walk, replacement policies	engine.js with first unit tests
5	Faults and processes	Demand paging, frame replacement, protection, ASIDs	Extended engine and tests
6–7	User interface (version 1)	Probe, bit surgery, datapath, timeline, structures	Single-page prototype
8	Analytics and policy lab	KPIs, charts, EAT calculator, policy comparison, Belady detection	Analytics features
9	Multi-page version (version 2)	Split into ten pages, shared session, navigation and transitions	Version 2 application
10	Testing	Engine tests, invariants, mutation testing, page tests, browser checks	Test results
11	Documentation	Learning notes, viva questions, printable report, README	User guide and report
12	Review and submission	Final fixes, case study and demonstration	Final submission

11. Expected Outcomes

- A working simulator that translates any virtual address on a configurable machine and explains every step at bit, stage and timeline level.
- Students can check textbook numericals instantly, for example VA 300 → PA 300 with a miss of 210 ns and a hit of 110 ns, and an EAT of 130 ns for h = 0.8.
- Classic phenomena can be demonstrated live: Belady's anomaly (FIFO gives 9 misses with 3 entries but 10 misses with 4 entries), LRU thrashing (a loop over five pages with a four-entry TLB gives 0% hits), conflict misses in a direct-mapped TLB, and ASID tagging versus flushing.
- Replacement policies and TLB sizes can be compared quantitatively on the same trace.
- Correctness is verified: all 96 engine tests (including 38 edge cases) and all 29 page-level tests pass.
- The tool can be reused as a teaching aid in lab sessions and for viva preparation, and it produces a printable report.

Table 11: *Selected test cases and results*

#	Input / setup	Expected result	Result
1	Default machine; VA 300 twice	VPN 1, offset 44, PA 300; miss then hit	Pass
2	VA 4095 and 4096	PA 4095; then a fault with no frame and no PA	Pass
3	0x12C, 0b100101100, 0o454, 0x1_2C	All equal 300	Pass
4	FIFO, 3 entries, pages 1 2 3 1 4 1	Page 1 evicted; outcomes M M M H M M	Pass
5	Belady string, FIFO with 3 and 4 entries	9 and 10 misses (anomaly)	Pass
6	Direct-mapped, 4 entries, pages 0 1 4 0 1	Pages 0 and 4 conflict in set 0: M M M M H	Pass
7	Write to a read-only page	Protection fault, no PA, page stays clean	Pass
8	Two processes, ASID mode, same VPN	M M H; 2 switches; 0 flushes	Pass
9	EAT with h = 0.8, t = 10, m = 100	130 ns	Pass
10	3,000 random multi-process demand-paging accesses	All invariants hold	Pass

Figures 4 to 7 show the simulator producing these outcomes.

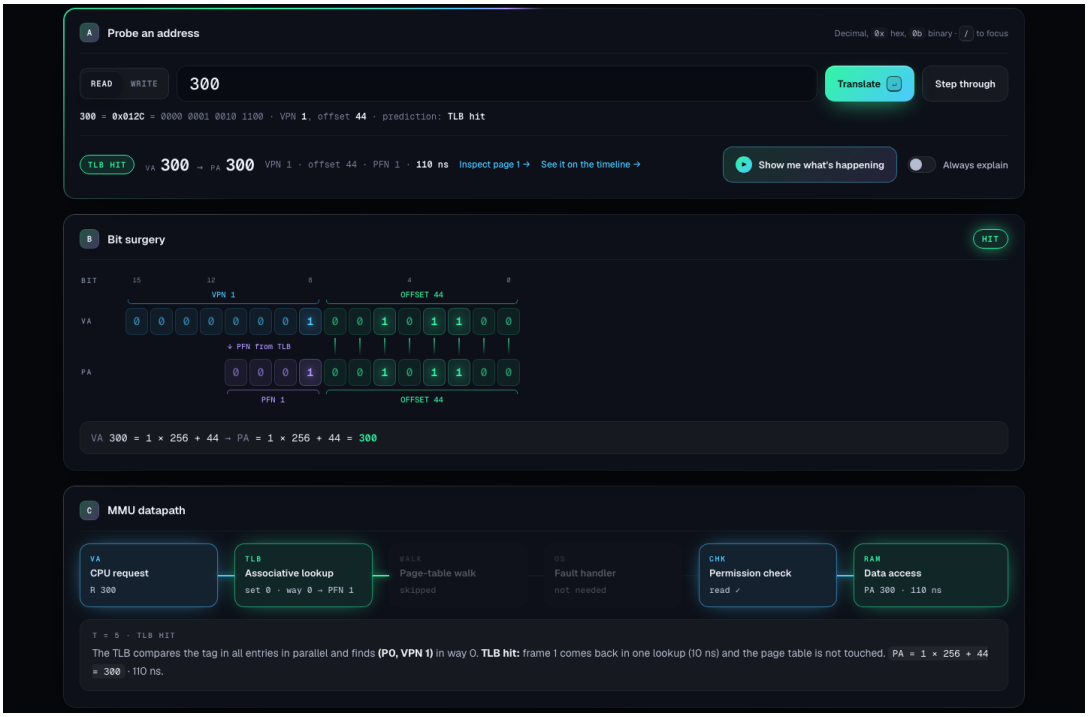


Figure 4: *Translate page: VA 300 is a TLB hit, giving PA 300 in 110 ns, with the bit-level view and the MMU datapath*

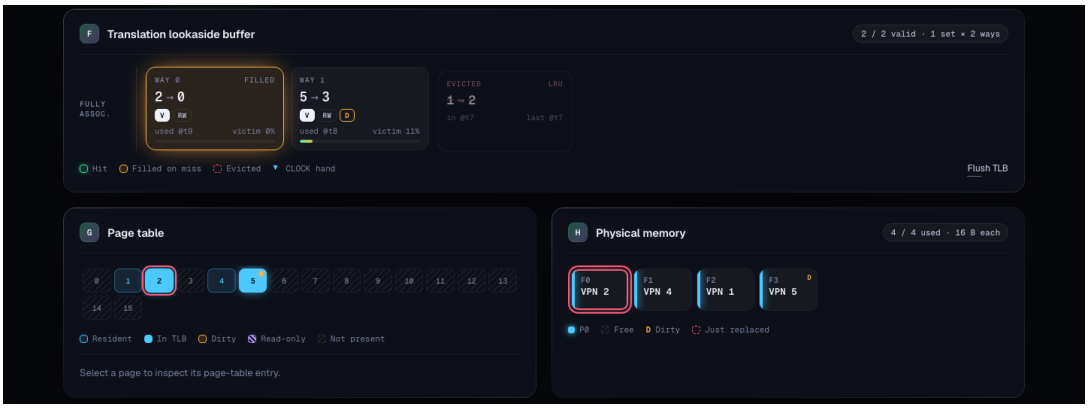


Figure 5: *Structures page: TLB ways, page-table map and physical frames during demand paging*

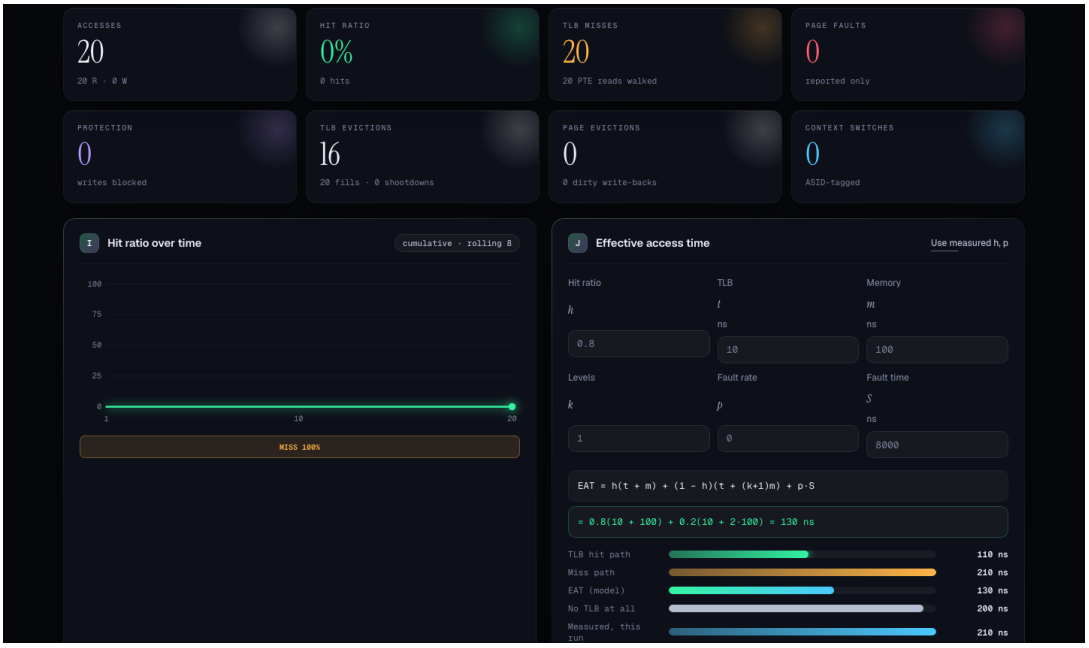


Figure 6: *Analytics page: LRU thrashing (0% hit ratio) and the EAT calculator (130 ns model versus 200 ns without a TLB)*

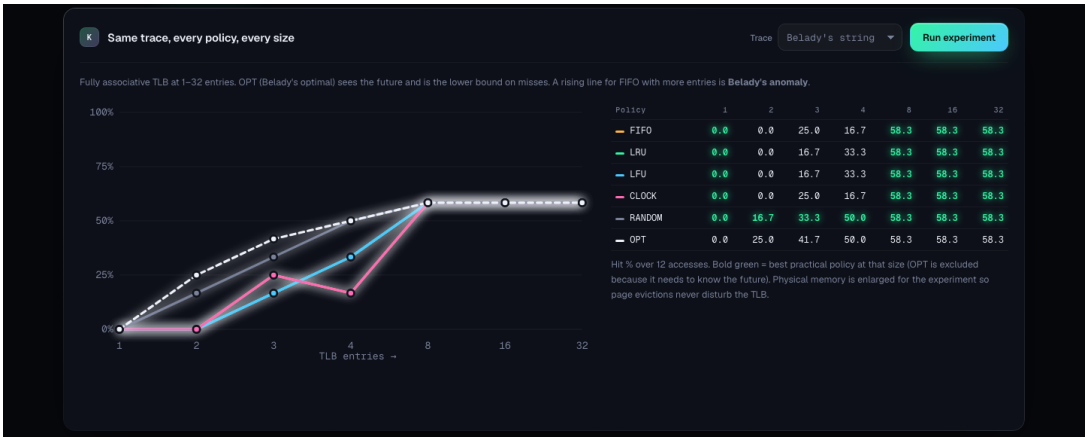


Figure 7: *Policy lab: hit ratio of six policies at seven TLB sizes on Belady's string; FIFO drops from 25.0% to 16.7% when the TLB grows from 3 to 4 entries*

12. Future Scope

- **Huge pages** (2 MiB and 1 GiB) with a separate large-page TLB.
- **Two-level TLB hierarchy** (L1 instruction and data TLBs with a shared L2 TLB) and split instruction/data TLBs.
- **Page-walk caches** and a more detailed timing model for the hardware page-table walker.
- **Multi-core simulation** with TLB shootdowns delivered by inter-processor interrupts.
- **Other page-table designs** such as inverted and hashed page tables, and working-set or page-fault-frequency frame allocation.
- **Real address traces** recorded from instrumented programs (for example with Valgrind or Intel Pin) and replayed in the simulator.
- **Nested paging** for virtual machines (guest and host page tables).

- **Exercise and quiz mode** with automatic grading for instructors.

13. Challenges and Ethical Considerations

13.1 Challenges

Table 12: *Challenges faced and how they were addressed*

Challenge	How it was addressed
Keeping every view consistent while allowing rewind and what-if changes	Event-sourced timeline: the state at time t is the replay of all earlier operations through a deterministic engine
Making the RANDOM policy reproducible	A seeded pseudo-random number generator, so every session replays identically
Large address spaces (a 32-bit VA with 4 KiB pages has 2^20 PTEs)	Typed arrays for page tables and paged views of 1,024 cells
Proving that the simulator is correct	Expected values derived by hand or from textbooks, invariant checks after every access, and mutation testing with deliberately planted bugs
Belady's string alone could not tell CLOCK apart from FIFO	Two extra CLOCK tests were added after mutation testing exposed the gap
Invalid input (out-of-range addresses, malformed hexadecimal)	A strict parser with specific error messages; invalid input never changes the state
Browser differences (native page transitions only in Chromium)	Feature detection with a JavaScript fade/slide fallback
Readability on small screens	Responsive layout tested at 375 pixels wide with no horizontal scrolling

13.2 Ethical considerations

- **Academic integrity:** concepts, formulas and worked examples taken from textbooks and lecture notes are cited in the references, and test expectations are taken from these published sources.
- **Accuracy and honesty:** the simulator is a teaching model, not a cycle-accurate MMU. Its limitations (no huge pages, no split TLBs, no page-walk caches, no multi-core shootdowns) are stated clearly so that students do not over-generalise.
- **Privacy:** everything runs locally in the browser. There are no accounts, no tracking and no server. Sessions are kept in localStorage, and share links keep their data in the URL fragment, which browsers do not send to servers.
- **Accessibility and inclusion:** keyboard shortcuts, ARIA live regions, support for reduced motion, text labels in addition to colours, light and dark themes, and support for low-end devices and phones.
- **Free and open access:** no paid software is required, and the project is released under the MIT licence so other institutions can reuse it.

- **Responsible use in learning:** the tool is meant to help students understand and check their answers, not to replace learning. Teachers can use the step-through mode and viva questions to test understanding.

14. Conclusion

This case study presented TLB/Scope, an interactive simulator for virtual-to-physical address translation using a Translation Lookaside Buffer. The simulator meets the requirement of Mini Project #45 and goes further. Every translation is shown at bit level, hardware-stage level and timeline level. The TLB organisation, replacement policy, page-table depth, fault handling and context-switch strategy can be changed and compared on the same workload. Performance is measured as well as modelled, and correctness is backed by 96 automated engine tests checked against textbook results, together with 29 page-level tests.

The project shows how a small hardware cache has a large effect on memory performance, and why choices such as replacement policy and associativity matter. By turning an invisible hardware process into something students can see, change and measure, TLB/Scope makes one of the harder topics in Operating Systems easier to learn and to teach. The future work listed in Section 12 would extend it towards the features of modern processors.

References

[1] A. Silberschatz, P. B. Galvin and G. Gagne, *Operating System Concepts*, 10th ed. Wiley, 2018.

[2] R. H. Arpaci-Dusseau and A. C. Arpaci-Dusseau, *Operating Systems: Three Easy Pieces*. Arpaci-Dusseau Books, 2018. Available: https://pages.cs.wisc.edu/~remzi/OSTEP/

[3] A. S. Tanenbaum and H. Bos, *Modern Operating Systems*, 4th ed. Pearson, 2015.

[4] J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed. Morgan Kaufmann, 2017.

[5] L. A. Belady, "A study of replacement algorithms for a virtual-storage computer," *IBM Systems Journal*, vol. 5, no. 2, pp. 78–101, 1966.

[6] N. Binkert et al., "The gem5 simulator," *ACM SIGARCH Computer Architecture News*, vol. 39, no. 2, pp. 1–7, 2011.

[7] D. Burger and T. M. Austin, "The SimpleScalar tool set, version 2.0," *ACM SIGARCH Computer Architecture News*, vol. 25, no. 3, pp. 13–25, 1997.

[8] J. Edler and M. D. Hill, "Dinero IV trace-driven uniprocessor cache simulator," University of Wisconsin–Madison, 1998.

[9] A. Bhattacharjee and D. Lustig, *Architectural and Operating System Support for Virtual Memory*. Morgan & Claypool, 2017.

[10] R. Ontko and A. Reeder, "MOSS Memory Management Simulator," companion software to *Modern Operating Systems*.

[11] Colorado State University, CS370 lecture notes on virtual memory. Available: https://www.cs.colostate.edu/~cs370/Fall25/lectures/10virtualmemoryL19.pdf

[12] Mozilla, MDN Web Docs (HTML, CSS and JavaScript). Available: https://developer.mozilla.org/